

CSES Advanced Problems – ICPC Trick Sheet

Recognition cues, reductions, and key invariants. Intended as a contest reference, not full proofs.

Dynamic Programming

Minimal Grid Path.

Process the grid diagonal by diagonal. Keep all cells that can belong to the current lexicographically minimal prefix. For the next diagonal, find the smallest reachable character and discard every successor with a larger character.

Graph Algorithms

Knight’s Tour.

At each step, try the knight move that leads to the square with the fewest available next moves. This avoids trapping yourself later, and if a path still fails, normal backtracking undoes it and tries the next-best move.

Range Queries

Visible Buildings Queries.

The trick is that the visible buildings from **a** are exactly the chain of next strictly greater elements to the right. Precompute those links with a monotonic stack, then use binary lifting to count how many links stay inside **[a,b]**.

Tree Algorithms

Fixed-Length Paths I.

Use centroid decomposition so every path is counted at the highest centroid it passes through. For each centroid, process child subtrees one by one: a node at depth **d** matches previously seen nodes at depth **k-d**, and only afterward is its subtree added to the counts, avoiding double-counting.

Fixed-Length Paths II.

Use **centroid decomposition** so every path is counted exactly once at the highest centroid it passes through. For each child subtree, count node depths and match them against depths from previously processed subtrees such that their sum lies in **[k1, k2]**, using suffix counts for $O(1)$ range queries.

Mathematics

System of Linear Equations.

Do Gaussian elimination normally, but modulo $10^9 + 7$: dividing by a pivot means multiplying by its modular inverse. If there are free variables, set them to 0 and back-substitute; if you get a row **0 = nonzero**, there is no solution.

String Algorithms

Inverse Suffix Array.

Process suffixes in the given suffix-array order and try to give consecutive suffixes the same first character. If removing that first character would reverse their required order (**rank[a+1] > rank[b+1]**), they must use different characters, so increment the current character.

Geometry

Robot Path.

Binary-search the first movement command whose path intersects any earlier part of the route, testing each prefix with sweep-line segment intersection. Ignore the shared endpoint between consecutive commands so normal turns are not counted as collisions, then find the exact earliest intersection distance on the first bad segment.

Advanced Techniques

Corner Subgrid Check.

For each letter, every row generates pairs of columns containing that letter; seeing the same pair twice means those two rows and columns form the required rectangle. If there are already more generated pairs than the $n(n-1)/2$ possible column-pairs, pigeonhole guarantees the answer is YES without enumerating them.

Sliding Window Problems

Sliding Window Advertisement.

For a bar of height **h**, its maximal interval **[l,r]** contains only bars of height at least **h**; as the window slides across this interval, the usable width first increases linearly, is constant, then decreases linearly. Represent those three area functions as lines and range-insert them into a Li Chao tree, then query every window start for the maximum.

Interactive Problems

Hidden Integer.

YES means $\text{mid} < x$, so **x** must be to the right; otherwise $x \leq \text{mid}$. We binary-search for the first value for which the answer is NO, which is exactly **x**.

Hidden Permutation.

Sort the indices using the interactive comparison as the merge-sort comparator. Since the hidden array is a permutation of **1..n**, the smallest index gets value 1, the next gets 2, etc., directly reconstructing the permutation.

K-th Highest Score.

Suppose exactly **x** Finnish and **k-x** Swedish coders belong to the global top **k**. As **x** increases, **F[x]** decreases while **S[k-x+1]** increases, so **F[x] > S[k-x+1]** is monotonic and its last true position gives the correct partition.

Permuted Binary Strings.

Give every original position *i* a unique binary ID (**i-1**) and ask one query for each bit of that ID. The returned bits at position *j* reconstruct the ID of *a_j*, so after at most 10 queries you know the entire permutation.

Colored Chairs.

Keep an interval whose endpoint colors are inconsistent with perfect alternation; therefore somewhere inside it there must be two consecutive equal colors. Query the midpoint and keep whichever half has the same inconsistency, reducing to one edge in $O(\log n)$ queries—at most $2 + \text{ceil}(\log_2(n-1)) \leq 20$.

Inversion Sorting.

The key is that reversing a segment of length *k* containing *x* inversions changes the global inversion count by:

$$\binom{k}{2} - 2x.$$

Reversing a segment flips the inversion status of every pair inside it, so the change in the reported global inversion count reveals exactly how many inversions that segment had. Prefix inversion differences give the inversion sequence of the hidden permutation; reconstruct it, then sort the now-known permutation directly.

Bitwise Operations

K Subset Xors.

The insight is the reduced xor basis: after eliminating each pivot bit from all higher-pivot vectors, coefficient masks 0, 1, 2, ... correspond directly to subset-xor values in increasing numerical order. Dependencies

in the original array simply make every distinct xor appear 2^{n-r} times, including zero.

All Subarray Xors.

A subarray XOR is $\text{pref}[r] \oplus \text{pref}[l-1]$, so we need every XOR produced by a pair of prefix XOR values. XOR convolution via FWHT finds all such pairs in $O(2^{20} \cdot 20)$; 0 needs special handling because pairing a prefix with itself does not correspond to a non-empty subarray.

Xor Pyramid Row.

Moving up *m* rows applies $(1+x)^m$; over XOR (mod 2), $(1+x)^{2^b} = 1+x^{2^b}$. Therefore, for every set bit 2^b of **m**, simply apply $a[i] \oplus a[i + 2^b]$, giving $O(n \log n)$ time and $O(n)$ memory.

SOS Bit Problem.

This is SOS DP / subset zeta transform. **sub[x]** counts all submasks of **x**, **sup[x]** counts all supermasks, and $x \& y \neq 0$ is obtained by subtracting from **n** all **y** contained in the bitwise complement of **x**. Complexity: $O(20 \cdot 2^{20} + n)$, appropriate for the CSES limits.

And Subset Count.

SOS DP finds how many array elements are supermasks of each mask, so there are $2^{\text{cnt}[\text{mask}]} - 1$ subsets whose AND contains that mask. Then Boolean-lattice Möbius inversion converts “AND contains mask” into “AND equals mask”.

Complexity: $O(n + 2^B B)$, where 2^B is the smallest power of two greater than *n*; here roughly $O(n \log n)$ with at most 2^{18} masks.

Construction Problems

Third Permutation.

Relabel values by their positions in **a**, turning **b** into a derangement **p**; if we write all cycles of **p** consecutively, shifting this order by 2 gives another permutation avoiding both the original position and **p[i]**. Mapping those shifted positions back through **a** produces **c**, and this always works for **n** ≥ 3 (while **n** = 2 is impossible).

Permutation Prime Sums.

For any current **r**, Bertrand’s postulate guarantees a prime **p** with $r < p \leq 2r$. Thus **l** = **p-r** ≥ 1 , and mapping **i** -> **p-i** simply permutes the entire interval **[l,r]** while making every corresponding sum equal to the prime **p**.

Distinct Sums Grid.

Start from a grid whose row sums form an arithmetic progression and whose column sums are consecutive; for *n* > 4 , exactly one row/column sum collision needs fixing. Two carefully chosen swaps destroy that collision without changing how many times each number occurs.

Filling Trominos.

After making one dimension divisible by 3, every solvable rectangle is composed from 3×2 / 2×3 blocks, except the odd×odd case, which uses one fixed 9×5 tiling and reduces the remainder to the easy cases. An odd 3×odd rectangle is impossible; these cases characterize the construction.

Grid Path Construction.

The trick is to use the exact feasibility rules for Hamiltonian paths in rectangular grids, then recursively reduce the grid by removing a 2-column strip or splitting it into two smaller solvable rectangles. For tiny rectangles, brute-force DFS handles the annoying special cases, so the main construction stays simple.

Advanced Graph Problems

Nearest Shops.

The trick is to run multi-source BFS from every shop, assigning each city to its closest shop. For a shop itself we must exclude it, so scan edges

connecting two different BFS regions: such an edge gives a candidate distance $\text{dist}[u] + 1 + \text{dist}[v]$ between their two shops.

Prüfer Code.

The key observation is that a node’s initial degree is $1 +$ the number of times it appears in the Prüfer code. Then simulate the encoding process in reverse, always taking the smallest current leaf with a min-heap, exactly matching CSES’s definition.

Fixed Length Walk Queries.

Duplicate every vertex into two states (v, parity) and run BFS, so $\text{dist}[v][0/1]$ gives the shortest even/odd-length walk to v. A query (a,b,x) is possible iff the shortest walk from a to b with parity $x \% 2$ has length $\leq x$, because any walk can be extended by 2 using an edge and immediately coming back.

MST Edge Check.

Run Kruskal by weight groups; an edge can belong to some MST exactly when its endpoints are not already connected using strictly lighter edges. Therefore, check every edge of the same weight before unioning any of them, then union the whole group

MST Edge Set Check.

Kruskal tells us that when handling edges of weight w, only components formed by edges with weight $< w$ matter; inside those contracted components, the query’s weight-w edges must not form a cycle. Use a rollback DSU to temporarily add each query’s edges of the same weight, reject on a cycle, then undo before testing the next query.

MST Edge Cost.

MST Edge Cost: build one MST with Kruskal. If an edge (u,v,w) is forced in, add it to the MST cycle and remove the maximum-weight edge on the MST path from u to v, so the answer is $\text{mst_cost} + w - \text{max_on_path}(u,v)$.

Tree Coin Collecting I.

For a query (a,b), the answer is $\text{dist}(a,b) + 2 * \text{minCoinDist}(v)$ over all vertices v on the path $a \rightarrow b$. Compute each vertex’s distance to the nearest coin with multi-source BFS, then answer the minimum value on a tree path using binary lifting.

Tree Coin Collecting II.

Isolate the minimal subtree containing every coin. For a query (a,b), only the branches from a and b to this fixed subtree can add new edges; the final walk traverses every required edge twice except the $a \rightarrow b$ path, which is traversed once.

Graph Coloring.

Each color class must be an independent set, so the problem becomes partitioning all vertices into the minimum number of independent subsets. Precompute which subsets are independent, then use subset DP; forcing each chosen subset to contain the lowest set bit avoids considering the same partition in many orders.

Bus Companies.

Model each bus company as an extra node: going city $\rightarrow$ company costs the ticket price, while company $\rightarrow$ city costs 0. Then ordinary Dijkstra on this bipartite graph gives the answer; the total number of connections is only $O(\sum k) \leq 2 \cdot 10^5$

Split into Two Paths.

In topological order, after covering the prefix, assume one path ends at the newest vertex i; the DP state is just where the other path can end. When processing i+1, either i $\rightarrow$ i+1 exists and all old states survive, or it doesn’t and all old states die, leaving at most the newly-created switched state—this lets us implement the nominal $dp[i][j]$ in $O(n + m)$.

Forbidden Cities.

The key is low-link values: after deleting c, every DFS-child subtree with $\text{low}[\text{child}] >= \text{tin}[c]$ becomes its own component; everything else belongs to the “parent side.” Binary lifting lets us find, for any queried vertex, which direct child-subtree of c contains it in $O(\log n)$.

New Flight Routes.

Condense the graph into SCCs, giving a DAG; the minimum number of new edges is $\max(\# \text{source SCCs}, \# \text{sink SCCs})$. The tricky part is constructing those edges without bad pairings, so the solution adds $\text{sink} \rightarrow \text{source}$ edges in verified batches and keeps only batches that reduce both counts exactly as expected.

Counting Problems

Filled Subgrid Count I.

The trick is a largest-square DP: $dp[j]$ is the largest monochromatic square ending at the current cell. If the current cell and its upper, left, and upper-left neighbors have the same letter, the size is $1 + \min(\text{up}, \text{left}, \text{diag})$; every size from $1..dp[j]$ gives one valid square ending there.

Filled Subgrid Count II.

$h[j]$ is the consecutive vertical run of the current letter ending at (i,j). Inside each contiguous same-letter segment of a row, every horizontal interval forms $\min(\text{height})$ rectangles ending on this row, and the monotonic stack computes the sum of those minima in amortized $O(1)$ per cell.

All Letter Subgrid Count I.

For a cell (i,j) used as the bottom-right corner, $dp[.][j][c]$ is the smallest side length of a square ending there that reaches some occurrence of letter c; if the current cell is not c, it is one plus the minimum of the values from above, left, and diagonal. The largest such value over all letters is the minimum side length containing every letter, so every larger square ending at the same cell is also valid.

All Letter Subgrid Count II.

Fix a pair of rows and compress each column into a 26-bit mask of letters appearing between those rows; then the problem becomes counting subarrays whose bitwise OR equals the full alphabet mask. Because OR has no inverse, maintain the sliding-window OR with two stacks storing prefix-ORs, giving amortized $O(1)$ push/pop and $O(n^3)$ total time.

Border Subgrid Count I.

For each cell, precompute how far equal letters extend in the two directions needed for it to serve as a top-left corner and as a bottom-right corner; then every valid square corresponds to a pair of cells on the same NW–SE diagonal whose reaches overlap enough. The opposite corner letters need no separate check, because if all four border runs reach the required length, they meet at the other corners and force the entire border to have one letter.

Border Subgrid Count II.

Fix the top and bottom rows; inside every maximal interval where both rows consist of the same letter, any columns whose entire vertical segment has that letter can serve as the left/right borders. As we scan such an interval, if cnt valid border columns have appeared before the current valid column, it creates $\text{cnt} + 1$ rectangles, giving an $O(n^3)$ solution with $O(n^2)$ preprocessing.

Raab Game II.

Fix one player’s order as $1, 2, \dots, n$; then the other player’s order becomes a permutation where wins/losses correspond to $p_i < i$ and $p_i > i$, while ties are fixed points. Count permutations with exactly a exceedance directions and b opposite directions using the 2D recurrence, choose which a + b values are non-fixed, then multiply by n! to restore the first player’s arbitrary order.

Counting Reorders.

Use inclusion–exclusion on the $c - 1$ possible bonds between identical copies of each character: choosing j bonds merges those copies into $c - j$ blocks and contributes with sign $(-1)^j$. DP combines these choices across the 26 letters, and for a total of J chosen bonds, the resulting blocks can be arranged in $(n - J)! / \prod (c_i - j_i)!$ ways.

Tournament Graph Distribution.

The key fact is that after contracting SCCs of a tournament, the resulting DAG is itself a tournament, so it must be a total order; all edges between consecutive SCC blocks therefore have forced directions. This lets us uniquely choose the first SCC, giving both the recurrence for strong[i] and the DP for the number of SCCs.

Collecting Numbers Distribution.

The number of collection rounds equals **1 plus the number of descents** in the position sequence $\text{pos}[1], \text{pos}[2], \dots, \text{pos}[n]$, so counting permutations by rounds is exactly counting permutations by descents. Those counts are Eulerian numbers, giving the recurrence $dp[i][k] = k * dp[i-1][k] + (i - k + 1) * dp[i-1][k-1]$.

Additional Problems I

Distinct Values Sum.

Maintain cur, the sum of distinct-value counts over all subarrays ending at the current position. When processing $x[i]$, exactly the suffixes starting after its previous occurrence gain one new distinct value, so do $\text{cur} += i - \text{last}[x[i]]$, then add cur to the global answer.

Distinct Values Splits.

Maintain a sliding window $[l..i]$ with all distinct values; then the last segment of any valid split ending at i may start at any position $j \in [l, i]$. Therefore $dp[i+1] = dp[l] + \dots + dp[i]$, and prefix sums make that transition $O(1)$, yielding $O(n)$ overall.

Beautiful Permutation II.

The trick is to greedily choose the smallest currently valid number while more than five numbers remain; with five remaining, a valid completion can always be arranged. Then enumerate those at most $5! = 120$ suffix permutations in lexicographic order and take the first valid one, giving the globally lexicographically smallest answer.

Bubble Sort Rounds II.

The key observation is that after k bubble-sort rounds, an element can move at most k positions to the left. Equivalently, when constructing the resulting array from left to right, position i receives the smallest unused element among original positions $0 \dots i + k$. This matches the bubble-sort process exactly, including duplicates if we break ties by original index.

Nearest Campsites I.

For each free campsite, split reserved campsites into the four quadrants around it. Inside one quadrant, Manhattan distance becomes a simple signed linear expression; an offline sweep + Fenwick tree finds the best reserved campsite in that quadrant in $O(\log n)$.

Nearest Campsites II.

For one quadrant, if a reserved point satisfies $x_r \leq x_q, y_r \leq y_q$, its distance is $(x_q + y_q) - (x_r + y_r)$, so we need the maximum $x_r + y_r$ among dominated points. Flip the signs of x and y in all four combinations to cover all four quadrants, and solve each dominance query with an x-sweep + Fenwick prefix maximum.

Subarray Sum Constraints.

Turn into equations on prefix sums. Let $p_i = x_1 + \dots + x_i$; then every constraint (l,r,s) becomes $p_r - p_{l-1} = s$, so we just need to assign consistent potentials in a weighted graph.

Treat prefix sums $p_0, \dots, p_n$ as graph vertices, with constraint (l,r,s) adding edges $p_r = p_{l-1} + s$ and $p_{l-1} = p_r - s$. BFS each component assigning potentials; any conflicting value means NO, otherwise recover $x_i = p_i - p_{i-1}$.

Water Containers Moves.

The crucial observation is that after every legal move, at least one container is empty or full, so all reachable states lie on the boundary of the (A,B) state grid. Thus there are only O(a+b) relevant states, and Dijkstra minimizes the total amount of water moved.

Water Containers Queries.

For each query, the target must fit in container A ($x \leq a$) and must be divisible by $\gcd(a,b)$. These are also sufficient by the standard two-jug reachability argument.

Every reachable amount of water in either container is a multiple of $\gcd(a,b)$, and every such amount up to that container's capacity is reachable. Since the final amount must specifically be in A, the condition is exactly $x \leq a$ && $x \% \gcd(a,b) == 0$.

Maximum Average Subarrays.

The key is to view prefix sums as points (i, S_i) : for endpoint i , the average of subarray $j + 1..i$ is exactly the slope from (j, S_j) to (i, S_i) . Only points on the lower convex hull can maximize this slope; binary-search the tangent, preferring the leftmost point on ties to obtain the longest subarray

Subsets with Fixed Average.

The key reduction is to replace each value by $x[i] - a$; then a subset has average a exactly when its transformed sum is 0. Since $n \leq 500$ and values are bounded by 500, a knapsack DP over possible sums is sufficient.

We can keep only nonnegative sums: process all positive transformed values before negative ones. Any state that becomes negative can never return to zero afterward, and sums larger than the total available magnitude on the opposite side can also be discarded.

Two Array Average.

The trick is binary search on the optimal average. For a candidate average x , subtract x from every element; then we only need the maximum transformed prefix sum from each array, and x is achievable iff their sum is nonnegative.

For prefixes of lengths i, j , average $\geq x$ is equivalent to

$$(A_i - xi) + (B_j - xj) \geq 0.$$

Since the two terms are independent, simply take the maximum transformed prefix from each array.

Permutation Subsequence.

The key reduction is: since both arrays are permutations, each value appears at most once, so map every value to its position in a . Then the LCS becomes the LIS of the positions of the elements of b that also exist in a , and we store parents to reconstruct the actual subsequence.

Letter Pair Move Game.

The key is to brute-force the tiny exceptional cases $n \leq 3$. For $n \geq 4$, every possible 8-cell configuration can be reduced to a sorted 8-cell configuration with the holes at its beginning; there are only $7 \cdot 2^6 = 448$ such states, so we precompute this with BFS and use it to finish the remaining suffix. This matches the known constructive approach for the problem, where solutions always exist for $n \geq 4$.

Additional Problems II

Bouncing Ball Steps.

Each coordinate is just a 1D walk bouncing between its two borders. A direction change happens at every multiple of $n-1$ or $m-1$; use inclusion-exclusion because reaching a corner changes both components simultaneously but counts as one direction change.

Bouncing Ball Cycle.

Unfold the reflections: each coordinate has period $2(n-1)$ or $2(m-1)$, so the ball returns to the corner after $2\text{lcm}(n-1, m-1)$ steps. If $g = \gcd(n-1, m-1)$, $A = (n-1)/g$, $B = (m-1)/g$, counting the cells identified by the reflected diagonals gives

cells = lcm($n-1, m-1$) - $\frac{(A-1)(B-1)}{2}$ + 1 .

Complexity: $O(\log \min(n, m))$ per test from gcd, $O(1)$ memory.

Knight Moves Queries.

Use the lower bounds $\lceil x/2 \rceil$ and $\lceil (x+y)/3 \rceil$, then fix parity since a knight flips square color every move. Because the board starts at $(1, 1)$ instead of extending infinitely, handle the three small exceptional displacements $(1, 0)$, $(1, 1)$, $(2, 2)$ separately.

K Subset Sums I.

The key is to turn negative values into nonnegative “toggle costs.” Let base be the sum of all negative elements; then every original subset sum is uniquely base + (subset sum of $|x_-|$), so we only need the k smallest subset sums of a sorted nonnegative array.

The heap trick generates every nonempty subset exactly once: from a subset whose largest chosen index is i , either add $a[i+1]$ or replace $a[i]$ with $a[i+1]$. Because a is sorted and nonnegative, popping the minimum heap element gives the subset sums in increasing order.

K Subset Sums II.

The key is to view all size- m combinations as nodes of an implicit min-heap. After sorting, the minimum combination is $[0, 1, \dots, m-1]$; every other combination has a unique parent obtained by decrementing its leftmost index that differs from the minimum, which gives each node at most two children.

Binary Subsequences.

The useful transformation is: let (a,b) encode the numbers of distinct subsequences associated with the two ending bits, shifted by +1. Appending a bit performs $(a,b) \rightarrow (a+b,b)$ or $(a,a+b)$, so reversing the process is exactly the subtractive Euclidean algorithm; therefore we enumerate coprime $a+b=n+2$ and choose the pair requiring the fewest Euclidean subtraction steps.

Grid Coloring II.

A clean way to view this is 2-SAT: every cell has exactly two possible new colors, and for every adjacent pair we forbid assignments where both choose the same color. With $n, m \leq 500$, there are at most 250,000 variables; the iterative Kosaraju implementation below avoids recursion-depth issues.

Treat the two colors different from a cell's original color as the two truth values of one boolean variable. For each adjacent pair and each color they could both choose, add the 2-SAT clause forbidding them from simultaneously choosing it; SCCs then give either a valid grid or IMPOSSIBLE.

Removing Digits II.

n can be as large as 10^{18} , so the usual $O(n)$ DP is impossible. The key observation is that an optimal solution always subtracts the largest digit currently present; then we accelerate that greedy process by recursively consuming whole decimal suffixes.

The minimum-step function is monotone, so among all allowed digits it is always optimal to subtract the largest one. Instead of simulating up to $10^{18}/9$ moves, solve(mx,n) processes an entire decimal suffix at once while remembering the largest digit mx available in the untouched prefix.

Replace with Difference.

The trick is that every sequence of differences leaves a value of the form $|\pm x_1 \pm x_2 \pm \dots \pm x_n|$, so reaching 0 requires splitting the array into two groups with equal sum. Since $n, x_i \leq 1000$, a bitset subset-sum finds such a partition efficiently; then we repeatedly cancel one value from each side, assigning the difference back to the larger side.

GCD Subsets.

The key is to count subsets whose elements are all divisible by k , then subtract subsets whose GCD is a larger multiple of k . Processing k from n down to 1 makes this a simple divisor-sieve DP in $O(n \log n)$.

Minimum Cost Pairs.

After sorting, an optimal pairing only needs adjacent elements, so the problem becomes finding the minimum-weight matching of every size on a path. The standard greedy + regret trick repeatedly takes the cheapest

active edge; if we later want both neighboring edges instead, the extra cost is left + right - chosen, so we contract those three edges into one new candidate.

Same Sum Subsets.

The key is a meet-in-the-middle subset-counting trick. The constraint $\sum x_i \leq 2^n - 2$ guarantees a collision by pigeonhole principle.

Let $F(s)$ be the number of non-empty subsets with sum at most s ; using MITM we can evaluate $F(s)$ in $O(2^{n/2})$. Find adjacent lo, hi with $F(lo) \leq lo$ and $F(hi) > hi$; since $hi = lo+1$, at least two subsets must have sum exactly hi , which we recover with another two-pointer pass.

Maximum Building II.

Process each row as the bottom of a histogram. Every horizontal interval is assigned uniquely to one occurrence of its minimum height; the number of intervals of each width containing that minimum forms a triangle/trapezoid, which can be added to all heights $1 \dots hi$ using 2D difference arrays.

Stick Difference.

If a stick of length a_i is split into c_i pieces, optimally those pieces have lengths

$$\left\lfloor \frac{a_i}{c_i} \right\rfloor \quad \text{and} \quad \left\lceil \frac{a_i}{c_i} \right\rceil.$$

Repeatedly splitting the stick with the largest current $\text{ceil}(a[i]/\text{cnt}[i])$ gives $hi[k]$, the smallest possible longest piece. Separately, merging the sequences $\text{floor}(a[i]/2)$, $\text{floor}(a[i]/3)$, ... with another max-heap gives the largest possible shortest piece.

For fixed k , the answer reduces to

$$\min_{0 \leq j \leq k} (hi[j] - \min(lo[j], best_min[k])).$$

Because both lo and hi are non-increasing, this becomes one moving boundary p plus a sliding-window minimum of $hi[j]-lo[j]$, handled by the deque.

Two Stacks Sorting.

Model each element as the interval during which it must stay inside a stack; two intervals that cross must be assigned to different stacks, so the problem becomes checking whether the interval-crossing graph is bipartite. Since that graph can be quadratic, sweep the interval endpoints and use a parity DSU to add only enough crossing constraints to preserve the coloring information.